message that is going to be sent. This creates severe key management problems, which has limited the use of the *One-time Pad* in commercial applications. However, the *One-time Pad* has been employed in military and diplomatic contexts, where unconditional security may be of great importance.

The historical development of cryptography has been to try to design cryptosystems where one key can be used to encrypt a relatively long string of plaintext (i.e., one key can be used to encrypt many messages) and still maintain some measure of computational security. Cryptosystems of this type include the *Data Encryption Standard* and the *Advanced Encryption Standard*, which we will discuss in the next chapter.

2.4 Entropy

In the previous section, we discussed the concept of perfect secrecy. We restricted our attention to the special situation where a key is used for only one encryption. We now want to look at what happens as more and more plaintexts are encrypted using the same key, and how likely a cryptanalyst will be able to carry out a successful ciphertext-only attack, given sufficient time.

The basic tool in studying this question is the idea of entropy, a concept from information theory introduced by Shannon in 1948. Entropy can be thought of as a mathematical measure of information or uncertainty, and is computed as a function of a probability distribution.

Suppose we have a discrete random variable $\mathbf{X}$ which takes values from a finite set X according to a specified probability distribution. What is the information gained by the outcome of an experiment which takes place according to this probability distribution? Equivalently, if the experiment has not (yet) taken place, what is the uncertainty about the outcome? This quantity is called the entropy of $\mathbf{X}$ and is denoted by $H(\mathbf{X})$.

These ideas may seem rather abstract, so let's look at a more concrete example. Suppose our random variable $\mathbf{X}$ represents the toss of a coin. As mentioned earlier, the associated probability distribution is $\mathbf{Pr}[heads] = \mathbf{Pr}[tails] = 1/2$. It would seem reasonable to say that the information, or entropy, of a coin toss is one bit, since we could encode *heads* by 1 and *tails* by 0, for example. In a similar fashion, the entropy of n independent coin tosses is n, since the n coin tosses can be encoded by a bitstring of length n.

As a slightly more complicated example, suppose we have a random variable $\mathbf{X}$ that takes on three possible values x_1, x_2, x_3 with probabilities $1/2, 1/4, 1/4$ respectively. Suppose we encode the three possible outcomes as follows: x_1 is encoded as 0, x_2 is encoded as 10, and x_3 is encoded as 11. (This is an example of a Huffman encoding; see Section 2.4.1). Then the (weighted) average number of bits in this encoding of $\mathbf{X}$ is

$$\frac{1}{2} \times 1 + \frac{1}{4} \times 2 + \frac{1}{4} \times 2 = \frac{3}{2}.$$

The above examples suggest that an event which occurs with probability 2^{-n} could perhaps be encoded as a bitstring of length n. More generally, we could plausibly imagine that an outcome occurring with probability p might be encoded by a bitstring of length approximately $-\log_2 p$. Given an arbitrary probability distribution, taking on the values $p_1, p_2, \ldots, p_n$ for a random variable $\mathbf{X}$, we take the weighted average of the quantities $-\log_2 p_i$ to be our measure of information. This motivates the following formal definition.

Definition 2.4: Suppose $\mathbf{X}$ is a discrete random variable which takes on values from a finite set X. Then, the *entropy* of the random variable $\mathbf{X}$ is defined to be the quantity

$$H(\mathbf{X}) = -\sum_{x \in X} \mathbf{Pr}[x] \log_2 \mathbf{Pr}[x].$$

REMARK Observe that $\log_2 y$ is undefined if $y = 0$. Hence, entropy is sometimes defined to be the relevant sum over all the non-zero probabilities. However, since $\lim_{y \to 0} y \log_2 y = 0$, there is no real difficulty with allowing $\mathbf{Pr}[x] = 0$ for some x's.

Also, we note that the choice of two as the base of the logarithms is arbitrary; another base would only change the value of the entropy by a constant factor. ▮

Note that if $|X| = n$ and $\mathbf{Pr}[x] = 1/n$ for all $x \in X$, then $H(\mathbf{X}) = \log_2 n$. Also, it is easy to see that $H(\mathbf{X}) \geq 0$ for any random variable $\mathbf{X}$, and $H(\mathbf{X}) = 0$ if and only if $\mathbf{Pr}[x_0] = 1$ for some $x_0 \in X$ and $\mathbf{Pr}[x] = 0$ for all $x \neq x_0$.

Let us look at the entropy of the various components of a cryptosystem. We can think of the key as being a random variable $\mathbf{K}$ that takes on values in $\mathcal{K}$, and hence we can compute the entropy $H(\mathbf{K})$. Similarly, we can compute entropies $H(\mathbf{P})$ and $H(\mathbf{C})$ of random variables associated with the plaintext and ciphertext, respectively.

To illustrate, we compute the entropies of the cryptosystem of Example 2.3.

Example 2.3 ***(Cont.)*** We compute as follows:

$$\begin{aligned} H(\mathbf{P}) &= -\frac{1}{4}\log_2\frac{1}{4} - \frac{3}{4}\log_2\frac{3}{4} \\ &= -\frac{1}{4}(-2) - \frac{3}{4}(\log_2 3 - 2) \\ &= 2 - \frac{3}{4}(\log_2 3) \\ &\approx 0.81. \end{aligned}$$

Similar calculations yield $H(\mathbf{K}) = 1.5$ and $H(\mathbf{C}) \approx 1.85$. □

2.4.1 Huffman Encodings

In this section, we discuss briefly the connection between entropy and Huffman encodings. As the results in this section are not relevant to the cryptographic applications of entropy, it may be skipped without loss of continuity. However, this discussion may serve to further motivate the concept of entropy.

We introduced entropy in the context of encodings of random events which occur according to a specified probability distribution. We first make these ideas more precise. As before, $\mathbf{X}$ is a random variable which takes on values from a finite set X and p is the associated probability distribution.

An *encoding* of $\mathbf{X}$ is any mapping

$$f : X \to \{0,1\}^*,$$

where $\{0,1\}^*$ denotes the set of all finite strings of 0's and 1's. Given a finite list (or string) of events $x_1 \cdots x_n$, where each $x_i \in X$, we can extend the encoding f in an obvious way by defining

$$f(x_1 \cdots x_n) = f(x_1) \,\|\, \cdots \,\|\, f(x_n),$$

where $\|$ denotes concatenation. In this way, we can think of f as a mapping

$$f : X^* \to \{0,1\}^*.$$

Now, suppose a string $x_1 \cdots x_n$ is produced by a *memoryless source*, such that each x_i in the string occurs according to a specified probability distribution on X. This means that the probability of any string $x_1 \cdots x_n$ is computed to be

$$\mathbf{Pr}[x_1 \cdots x_n] = \mathbf{Pr}[x_1] \times \cdots \times \mathbf{Pr}[x_n].$$

REMARK The string $x_1 \cdots x_n$ need not consist of distinct values, since the source is memoryless. As a simple example, consider a sequence of n tosses of a fair coin. If we encode "*heads*" as "1" and "*tails*" as "0," then every binary string of length n corresponds to a sequence of n coin tosses. ■

Now, given that we are going to encode strings using the mapping f, it is important that we are able to decode in an unambiguous fashion. Thus it should be the case that the encoding f is injective.

Example 2.4 Suppose $X = \{a, b, c, d\}$, and consider the following three encodings:

$f(a)$	=	1	$f(b)$	=	10	$f(c)$	=	100	$f(d)$	=	1000
$g(a)$	=	0	$g(b)$	=	10	$g(c)$	=	110	$g(d)$	=	111
$h(a)$	=	0	$h(b)$	=	01	$h(c)$	=	10	$h(d)$	=	11

It can be seen that f and g are injective encodings, but h is not. Any encoding using f can be decoded by starting at the end and working backwards: every time a 1 is encountered, it signals the beginning of the current element.

An encoding using g can be decoded by starting at the beginning and proceeding sequentially. At any point where we have a substring that is an encoding of a, b, c, or d, we decode it and chop off the substring. For example, given the string 10101110, we decode 10 to b, then 10 to b, then 111 to d, and finally 0 to a. So the decoded string is $bbda$.

To see that h is not injective, it suffices to give an example:

$$h(ac) = h(ba) = 010.$$

□

From the point of view of ease of decoding, we would prefer the encoding g to f. This is because decoding can be done sequentially from beginning to end if g is used, so no memory is required. The property that allows the simple sequential decoding of g is called the prefix-free property. (An encoding g is a *prefix-free encoding* if there do not exist two elements $x, y \in X$, and a string $z \in \{0,1\}^*$ such that $g(x) = g(y) \,\|\, z$.)

The discussion to this point has not involved entropy. Not surprisingly, entropy is related to the efficiency of an encoding. We will measure the efficiency of an encoding f as we did before: it is the weighted average length (denoted by $\ell(f)$) of an encoding of an element of $\mathbf{X}$. So we have the following definition:

$$\ell(f) = \sum_{x \in X} \mathbf{Pr}[x]\,|f(x)|,$$

where $|y|$ denotes the length of a string y.

Now, our fundamental problem is to find an injective encoding, f, that minimizes $\ell(f)$. There is a well-known algorithm, known as Huffman's algorithm,

that accomplishes this goal. Moreover, the encoding f produced by Huffman's algorithm is prefix-free, and

$$H(\mathbf{X}) \leq \ell(f) < H(\mathbf{X}) + 1.$$

Thus, the value of the entropy of $\mathbf{X}$ provides a close estimate to the average length of the optimal injective encoding.

We will not prove the results stated above, but we will give a short, informal description of Huffman's algorithm. Huffman's algorithm begins with the probability distribution on the set X, and the code of each element is initially empty. In each iteration, the two elements having lowest probability are combined into one element having as its probability the sum of the two smaller probabilities. The element having lowest probability is assigned the value "0" and the element having next lowest probability is assigned the value "1." When only one element remains, the coding for each $x \in X$ can be constructed by following the sequence of elements "backwards" from the final element to the initial element x.

This is easily illustrated with an example.

Example 2.5 Suppose $X = \{a, b, c, d, e\}$ has the following probability distribution: $\mathbf{Pr}[a] = .05$, $\mathbf{Pr}[b] = .10$, $\mathbf{Pr}[c] = .12$, $\mathbf{Pr}[d] = .13$ and $\mathbf{Pr}[e] = .60$. Huffman's algorithm would proceed as indicated in the following table:

a	b	c	d	e
.05	.10	.12	.13	.60
0	1			
.15		.12	.13	.60
		0	1	
.15		.25		.60
0		1		
.40				.60
0				1
1.0				

This leads to the following encodings:

x	$f(x)$
a	000
b	001
c	010
d	011
e	1

Thus, the average length encoding is

$$\ell(f) = .05 \times 3 + .10 \times 3 + .12 \times 3 + .13 \times 3 + .60 \times 1$$
$$= 1.8.$$

Compare this to the entropy:

$$H(\mathbf{X}) = .2161 + .3322 + .3671 + .3842 + .4422$$
$$= 1.7402.$$

It is seen that the average length encoding is very close to the entropy. □

2.5 Properties of Entropy

In this section, we prove some fundamental results concerning entropy. First, we state a fundamental result, known as Jensen's inequality, that will be very useful to us. Jensen's inequality involves concave functions, which we now define.

Definition 2.5: A real-valued function f is a *concave function* on an interval I if

$$f\left(\frac{x+y}{2}\right) \geq \frac{f(x)+f(y)}{2}$$

for all $x, y \in I$. f is a *strictly concave function* on an interval I if

$$f\left(\frac{x+y}{2}\right) > \frac{f(x)+f(y)}{2}$$

for all $x, y \in I$, $x \neq y$.

Here is Jensen's inequality, which we state without proof.

THEOREM 2.5 ***(Jensen's inequality)*** *Suppose f is a continuous strictly concave function on the interval I. Suppose further that*

$$\sum_{i=1}^{n} a_i = 1$$

and $a_i > 0$, $1 \leq i \leq n$. Then

$$\sum_{i=1}^{n} a_i f(x_i) \leq f\left(\sum_{i=1}^{n} a_i x_i\right),$$

where $x_i \in I$, $1 \leq i \leq n$. Further, equality occurs if and only if $x_1 = \cdots = x_n$.

We now proceed to derive several results on entropy. In the next theorem, we make use of the fact that the function $\log_2 x$ is strictly concave on the interval $(0, \infty)$. (In fact, this follows easily from elementary calculus since the second derivative of the logarithm function is negative on the interval $(0, \infty)$.)

THEOREM 2.6 *Suppose* $\mathbf{X}$ *is a random variable having a probability distribution which takes on the values* $p_1, p_2, \ldots, p_n$, *where* $p_i > 0$, $1 \le i \le n$. *Then* $H(\mathbf{X}) \le \log_2 n$, *with equality if and only if* $p_i = 1/n$, $1 \le i \le n$.

PROOF Applying Jensen's inequality, we have the following:

$$\begin{aligned} H(\mathbf{X}) &= -\sum_{i=1}^{n} p_i \log_2 p_i \\ &= \sum_{i=1}^{n} p_i \log_2 \frac{1}{p_i} \\ &\le \log_2 \sum_{i=1}^{n} \left(p_i \times \frac{1}{p_i} \right) \\ &= \log_2 n. \end{aligned}$$

Further, equality occurs if and only if $p_i = 1/n$, $1 \le i \le n$. ∎

THEOREM 2.7 $H(\mathbf{X}, \mathbf{Y}) \le H(\mathbf{X}) + H(\mathbf{Y})$, *with equality if and only if* $\mathbf{X}$ *and* $\mathbf{Y}$ *are independent random variables.*

PROOF Suppose $\mathbf{X}$ takes on values x_i, $1 \le i \le m$, and $\mathbf{Y}$ takes on values y_j, $1 \le j \le n$. Denote $p_i = \mathbf{Pr}[\mathbf{X} = x_i]$, $1 \le i \le m$, and $q_j = \mathbf{Pr}[\mathbf{Y} = y_j]$, $1 \le j \le n$. Then define $r_{ij} = \mathbf{Pr}[\mathbf{X} = x_i, \mathbf{Y} = y_j]$, $1 \le i \le m$, $1 \le j \le n$ (this is the joint probability distribution).

Observe that

$$p_i = \sum_{j=1}^{n} r_{ij}$$

$(1 \le i \le m)$, and

$$q_j = \sum_{i=1}^{m} r_{ij}$$

$(1 \le j \le n)$. We compute as follows:

$$\begin{aligned} H(\mathbf{X}) - H(\mathbf{Y}) &= -\left(\sum_{i=1}^{m} p_i \log_2 p_i + \sum_{j=1}^{n} q_j \log_2 q_j \right) \\ &= -\left(\sum_{i=1}^{m} \sum_{j=1}^{n} r_{ij} \log_2 p_i + \sum_{j=1}^{n} \sum_{i=1}^{m} r_{ij} \log_2 q_j \right) \\ &= -\sum_{i=1}^{m} \sum_{j=1}^{n} r_{ij} \log_2 p_i q_j. \end{aligned}$$

On the other hand,

$$H(\mathbf{X}, \mathbf{Y}) = -\sum_{i=1}^{m} \sum_{j=1}^{n} r_{ij} \log_2 r_{ij}.$$

Combining, we obtain the following:

$$\begin{aligned} H(\mathbf{X}, \mathbf{Y}) - H(\mathbf{X}) - H(\mathbf{Y}) &= \sum_{i=1}^{m} \sum_{j=1}^{n} r_{ij} \log_2 \frac{1}{r_{ij}} + \sum_{i=1}^{m} \sum_{j=1}^{n} r_{ij} \log_2 p_i q_j \\ &= \sum_{i=1}^{m} \sum_{j=1}^{n} r_{ij} \log_2 \frac{p_i q_j}{r_{ij}} \\ &\le \log_2 \sum_{i=1}^{m} \sum_{j=1}^{n} p_i q_j \\ &= \log_2 1 \\ &= 0. \end{aligned}$$

(In the above computations, we apply Jensen's inequality, using the fact that the r_{ij}'s are positive real numbers that sum to 1.)

We can also say when equality occurs: it must be the case that there is a constant c such that $p_i q_j / r_{ij} = c$ for all i, j. Using the fact that

$$\sum_{j=1}^{n} \sum_{i=1}^{m} r_{ij} = \sum_{j=1}^{n} \sum_{i=1}^{m} p_i q_j = 1,$$

it follows that $c = 1$. Hence, equality occurs if and only if $r_{ij} = p_i q_j$, i.e., if and only if

$$\mathbf{Pr}[\mathbf{X} = x_i, \mathbf{Y} = y_j] = \mathbf{Pr}[\mathbf{X} = x_i] \mathbf{Pr}[\mathbf{Y} = y_j],$$

$1 \le i \le m$, $1 \le j \le n$. But this says that $\mathbf{X}$ and $\mathbf{Y}$ are independent. ∎

We next define the idea of conditional entropy.

Definition 2.6: Suppose $\mathbf{X}$ and $\mathbf{Y}$ are two random variables. Then for any fixed value y of $\mathbf{Y}$, we get a (conditional) probability distribution on X; we denote the associated random variable by $\mathbf{X}|y$. Clearly,

$$H(\mathbf{X}|y) = -\sum_x \mathbf{Pr}[x|y] \log_2 \mathbf{Pr}[x|y].$$

We define the *conditional entropy*, denoted $H(\mathbf{X}|\mathbf{Y})$, to be the weighted average (with respect to the probabilities $\mathbf{Pr}[y]$) of the entropies $H(\mathbf{X}|y)$ over all possible values y. It is computed to be

$$H(\mathbf{X}|\mathbf{Y}) = -\sum_y \sum_x \mathbf{Pr}[y]\mathbf{Pr}[x|y] \log_2 \mathbf{Pr}[x|y].$$

The conditional entropy measures the average amount of information about $\mathbf{X}$ that is not revealed by $\mathbf{Y}$.

The next two results are straightforward; we leave the proofs as exercises.

THEOREM 2.8 $H(\mathbf{X}, \mathbf{Y}) = H(\mathbf{Y}) + H(\mathbf{X}|\mathbf{Y})$.

COROLLARY 2.9 $H(\mathbf{X}|\mathbf{Y}) \leq H(\mathbf{X})$, *with equality if and only if* $\mathbf{X}$ *and* $\mathbf{Y}$ *are independent.*

2.6 Spurious Keys and Unicity Distance

In this section, we apply the entropy results we have proved to cryptosystems. First, we show a fundamental relationship exists among the entropies of the components of a cryptosystem. The conditional entropy $H(\mathbf{K}|\mathbf{C})$ is called the *key equivocation*; it is a measure of the amount of uncertainty of the key remaining when the ciphertext is known.

THEOREM 2.10 *Let* $(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$ *be a cryptosystem. Then*

$$H(\mathbf{K}|\mathbf{C}) = H(\mathbf{K}) + H(\mathbf{P}) - H(\mathbf{C}).$$

PROOF First, observe that $H(\mathbf{K}, \mathbf{P}, \mathbf{C}) = H(\mathbf{C}|\mathbf{K}, \mathbf{P}) + H(\mathbf{K}, \mathbf{P})$. Now, the key and plaintext determine the ciphertext uniquely, since $y = e_K(x)$. This implies that $H(\mathbf{C}|\mathbf{K}, \mathbf{P}) = 0$. Hence, $H(\mathbf{K}, \mathbf{P}, \mathbf{C}) = H(\mathbf{K}, \mathbf{P})$. But $\mathbf{K}$ and $\mathbf{P}$ are independent, so $H(\mathbf{K}, \mathbf{P}) = H(\mathbf{K}) + H(\mathbf{P})$. Hence,

$$H(\mathbf{K}, \mathbf{P}, \mathbf{C}) = H(\mathbf{K}, \mathbf{P}) = H(\mathbf{K}) + H(\mathbf{P}).$$

In a similar fashion, since the key and ciphertext determine the plaintext uniquely (i.e., $x = d_K(y)$), we have that $H(\mathbf{P}|\mathbf{K}, \mathbf{C}) = 0$ and hence $H(\mathbf{K}, \mathbf{P}, \mathbf{C}) = H(\mathbf{K}, \mathbf{C})$.

Now, we compute as follows:

$$\begin{aligned} H(\mathbf{K}|\mathbf{C}) &= H(\mathbf{K}, \mathbf{C}) - H(\mathbf{C}) \\ &= H(\mathbf{K}, \mathbf{P}, \mathbf{C}) - H(\mathbf{C}) \\ &= H(\mathbf{K}) + H(\mathbf{P}) - H(\mathbf{C}), \end{aligned}$$

giving the desired formula. ∎

Let us return to Example 2.3 to illustrate this result.

Example 2.3 (Cont.) We have already computed $H(\mathbf{P}) \approx 0.81$, $H(\mathbf{K}) = 1.5$ and $H(\mathbf{C}) \approx 1.85$. Theorem 2.10 tells us that $H(\mathbf{K}|\mathbf{C}) \approx 1.5 + 0.81 - 1.85 \approx 0.46$. This can be verified directly by applying the definition of conditional entropy, as follows. First, we need to compute the probabilities $\mathbf{Pr}[\mathbf{K} = K_i|\mathbf{y} = j]$, $1 \leq i \leq 3$, $1 \leq j \leq 4$. This can be done using Bayes' theorem, and the following values result:

$$\begin{array}{lll} \mathbf{Pr}[K_1|1] = 1 & \mathbf{Pr}[K_2|1] = 0 & \mathbf{Pr}[K_3|1] = 0 \\ \mathbf{Pr}[K_1|2] = \frac{6}{7} & \mathbf{Pr}[K_2|2] = \frac{1}{7} & \mathbf{Pr}[K_3|2] = 0 \\ \mathbf{Pr}[K_1|3] = 0 & \mathbf{Pr}[K_2|3] = \frac{3}{4} & \mathbf{Pr}[K_3|3] = \frac{1}{4} \\ \mathbf{Pr}[K_1|4] = 0 & \mathbf{Pr}[K_2|4] = 0 & \mathbf{Pr}[K_3|4] = 1. \end{array}$$

Now we compute

$$H(\mathbf{K}|\mathbf{C}) = \frac{1}{8} \times 0 + \frac{7}{16} \times 0.59 + \frac{1}{4} \times 0.81 + \frac{3}{16} \times 0 = 0.46,$$

agreeing with the value predicted by Theorem 2.10. ∎

Suppose $(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$ is the cryptosystem being used, and a string of plaintext

$$x_1 x_2 \cdots x_n$$

is encrypted with one key, producing a string of ciphertext

$$y_1 y_2 \cdots y_n.$$

Recall that the basic goal of the cryptanalyst is to determine the key. We are looking at ciphertext-only attacks, and we assume that Oscar has infinite computational resources. We also assume that Oscar knows that the plaintext is a "natural" language, such as English. In general, Oscar will be able to rule out certain keys, but many "possible" keys may remain, only one of which is the correct key. The remaining possible, but incorrect, keys are called *spurious keys*.

For example, suppose Oscar obtains the ciphertext string $WNAJW$, which has been obtained by encryption using a shift cipher. It is easy to see that there are only two "meaningful" plaintext strings, namely *river* and *arena*, corresponding respectively to the possible encryption keys F $(= 5)$ and W $(= 22)$. Of these two keys, one will be the correct key and the other will be spurious. (Actually, it is moderately difficult to find a ciphertext of length 5 for the *Shift Cipher* that has two meaningful decryptions; the reader might search for other examples.)

Our goal is to prove a bound on the expected number of spurious keys. First, we have to define what we mean by the entropy (per letter) of a natural language L, which we denote H_L. H_L should be a measure of the average information per letter in a "meaningful" string of plaintext. (Note that a random string of alphabetic characters would have entropy (per letter) equal to $\log_2 26 \approx 4.70$.) As a "first-order" approximation to H_L, we could take $H(\mathbf{P})$. In the case where L is the English language, we get $H(\mathbf{P}) \approx 4.19$ by using the probability distribution given in Table 1.1.

Of course, successive letters in a language are not independent, and correlations among successive letters reduce the entropy. For example, in English, the letter "Q" is always followed by the letter "U." For a "second-order" approximation, we would compute the entropy of the probability distribution of all digrams and then divide by 2. In general, define $\mathbf{P}^n$ to be the random variable that has as its probability distribution that of all n-grams of plaintext. We make use of the following definitions.

Definition 2.7: Suppose L is a natural language. The *entropy* of L is defined to be the quantity

$$H_L = \lim_{n \to \infty} \frac{H(\mathbf{P}^n)}{n}$$

and the *redundancy* of L is defined to be

$$R_L = 1 - \frac{H_L}{\log_2 |\mathcal{P}|}.$$

REMARK H_L measures the entropy per letter of the language L. A random language would have entropy $\log_2 |\mathcal{P}|$. So the quantity R_L measures the fraction of "excess characters," which we think of as redundancy.

In the case of the English language, a tabulation of a large number of digrams and their frequencies would produce an estimate for $H(\mathbf{P}^2)$. $H(\mathbf{P}^2)/2 \approx 3.90$ is one estimate obtained in this way. One could continue, tabulating trigrams, etc. and thus obtain an estimate for H_L. In fact, various experiments have yielded the empirical result that $1.0 \leq H_L \leq 1.5$. That is, the average information content in English is something like one to one-and-a-half bits per letter!

Using 1.25 as our estimate of H_L gives a redundancy of about 0.75. This means that the English language is 75% redundant! (This is not to say that one can arbitrarily remove three out of every four letters from English text and hope to still be able to read it. What it does mean is that it is possible to find a Huffman encoding of n-grams, for a large enough value of n, which will compress English text to about one quarter of its original length.)

Given probability distributions on $\mathcal{K}$ and $\mathcal{P}^n$, we can define the induced probability distribution on $\mathcal{C}^n$, the set of n-grams of ciphertext (we already did this in the case $n = 1$). We have defined $\mathbf{P}^n$ to be a random variable representing an n-gram of plaintext. Similarly, define $\mathbf{C}^n$ to be a random variable representing an n-gram of ciphertext.

Given $\mathbf{y} \in \mathbf{C}^n$, define

$$K(\mathbf{y}) = \{K \in \mathcal{K} : \exists \mathbf{x} \in \mathcal{P}^n \text{ such that } \mathbf{Pr}[\mathbf{x}] > 0 \text{ and } e_K(\mathbf{x}) = \mathbf{y}\}.$$

That is, $K(\mathbf{y})$ is the set of keys K for which $\mathbf{y}$ is the encryption of a meaningful string of plaintext of length n, i.e., the set of "possible" keys, given that $\mathbf{y}$ is the ciphertext. If $\mathbf{y}$ is the observed string of ciphertext, then the number of spurious keys is $|K(\mathbf{y})| - 1$, since only one of the "possible" keys is the correct key. The average number of spurious keys (over all possible ciphertext strings of length n) is denoted by $\overline{s}_n$. Its value is computed to be

$$\begin{aligned}
\overline{s}_n &= \sum_{\mathbf{y} \in \mathcal{C}^n} \mathbf{Pr}[\mathbf{y}](|K(\mathbf{y})| - 1) \\
&= \sum_{\mathbf{y} \in \mathcal{C}^n} \mathbf{Pr}[\mathbf{y}]|K(\mathbf{y})| - \sum_{\mathbf{y} \in \mathcal{C}^n} \mathbf{Pr}[\mathbf{y}] \\
&= \sum_{\mathbf{y} \in \mathcal{C}^n} \mathbf{Pr}[\mathbf{y}]|K(\mathbf{y})| - 1.
\end{aligned}$$

From Theorem 2.10, we have that

$$H(\mathbf{K}|\mathbf{C}^n) = H(\mathbf{K}) + H(\mathbf{P}^n) - H(\mathbf{C}^n).$$

Also, we can use the estimate

$$H(\mathbf{P}^n) \approx nH_L = n(1 - R_L)\log_2 |\mathcal{P}|,$$

provided n is reasonably large. Certainly,

$$H(\mathbf{C}^n) \leq n \log_2 |\mathcal{C}|.$$

Then, if $|\mathcal{C}| = |\mathcal{P}|$, it follows that

$$H(\mathbf{K}|\mathbf{C}^n) \geq H(\mathbf{K}) - nR_L \log_2 |\mathcal{P}|. \tag{2.2}$$

Next, we relate the quantity $H(\mathbf{K}|\mathbf{C}^n)$ to the number of spurious keys, $\overline{s}_n$. We compute as follows:

$$\begin{aligned} H(\mathbf{K}|\mathbf{C}^n) &= \sum_{\mathbf{y} \in \mathcal{C}^n} \mathbf{Pr}[\mathbf{y}] H(\mathbf{K}|\mathbf{y}) \\ &\leq \sum_{\mathbf{y} \in \mathcal{C}^n} \mathbf{Pr}[\mathbf{y}] \log_2 |K(\mathbf{y})| \\ &\leq \log_2 \sum_{\mathbf{y} \in \mathcal{C}^n} \mathbf{Pr}[\mathbf{y}] |K(\mathbf{y})| \\ &= \log_2(\overline{s}_n + 1), \end{aligned}$$

where we apply Jensen's inequality (Theorem 2.5) with $f(x) = \log_2 x$. Thus we obtain the inequality

$$H(\mathbf{K}|\mathbf{C}^n) \leq \log_2(\overline{s}_n + 1). \tag{2.3}$$

Combining the two inequalities (2.2) and (2.3), we get that

$$\log_2(\overline{s}_n + 1) \geq H(\mathbf{K}) - nR_L \log_2 |\mathcal{P}|.$$

In the case where keys are chosen equiprobably (which maximizes $H(\mathbf{K})$), we have the following result.

THEOREM 2.11 *Suppose $(\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D})$ is a cryptosystem where $|\mathcal{C}| = |\mathcal{P}|$ and keys are chosen equiprobably. Let R_L denote the redundancy of the underlying language. Then given a string of ciphertext of length n, where n is sufficiently large, the expected number of spurious keys, $\overline{s}_n$, satisfies*

$$\overline{s}_n \geq \frac{|\mathcal{K}|}{|\mathcal{P}|^{nR_L}} - 1.$$

The quantity $|\mathcal{K}|/|\mathcal{P}|^{nR_L} - 1$ approaches 0 exponentially quickly as n increases. Also, note that the estimate may not be accurate for small values of n, especially since $H(\mathbf{P}^n)/n$ may not be a good estimate for H_L if n is small.

We have one more concept to define.

Definition 2.8: The *unicity distance* of a cryptosystem is defined to be the value of n, denoted by n_0, at which the expected number of spurious keys becomes zero; i.e., the average amount of ciphertext required for an opponent to be able to uniquely compute the key, given enough computing time.

If we set $\overline{s}_n = 0$ in Theorem 2.11 and solve for n, we get an estimate for the unicity distance, namely

$$n_0 \approx \frac{\log_2 |\mathcal{K}|}{R_L \log_2 |\mathcal{P}|}.$$

As an example, consider the *Substitution Cipher*. In this cryptosystem, $|\mathcal{P}| = 26$ and $|\mathcal{K}| = 26!$. If we take $R_L = 0.75$, then we get an estimate for the unicity distance of

$$n_0 \approx 88.4/(0.75 \times 4.7) \approx 25.$$

This suggests that, given a ciphertext string of length at least 25, (usually) a unique decryption is possible.

2.7 Product Cryptosystems

Another innovation introduced by Shannon in his 1949 paper was the idea of combining cryptosystems by forming their "product." This idea has been of fundamental importance in the design of present-day cryptosystems such as the *Advanced Encryption Standard*, which we study in the next chapter.

For simplicity, we will confine our attention in this section to cryptosystems in which $\mathcal{C} = \mathcal{P}$; a cryptosystem of this type is called an *endomorphic cryptosystem*. Suppose $\mathbf{S}_1 = (\mathcal{P}, \mathcal{P}, \mathcal{K}_1, \mathcal{E}_1, \mathcal{D}_1)$ and $\mathbf{S}_2 = (\mathcal{P}, \mathcal{P}, \mathcal{K}_2, \mathcal{E}_2, \mathcal{D}_2)$ are two endomorphic cryptosystems which have the same plaintext (and ciphertext) spaces. Then the *product cryptosystem* of $\mathbf{S}_1$ and $\mathbf{S}_2$, denoted by $\mathbf{S}_1 \times \mathbf{S}_2$, is defined to be the cryptosystem

$$(\mathcal{P}, \mathcal{P}, \mathcal{K}_1 \times \mathcal{K}_2, \mathcal{E}, \mathcal{D}).$$

A key of the product cryptosystem has the form $K = (K_1, K_2)$, where $K_1 \in \mathcal{K}_1$ and $K_2 \in \mathcal{K}_2$. The encryption and decryption rules of the product cryptosystem are defined as follows: For each $K = (K_1, K_2)$, we have an encryption rule e_K defined by the formula

$$e_{(K_1,K_2)}(x) = e_{K_2}(e_{K_1}(x)),$$

and a decryption rule defined by the formula

$$d_{(K_1,K_2)}(y) = d_{K_1}(d_{K_2}(y)).$$

That is, we first encrypt x with e_{K_1}, and then "re-encrypt" the resulting ciphertext with e_{K_2}. Decrypting is similar, but it must be done in the reverse order:

$$\begin{aligned} d_{(K_1,K_2)}(e_{(K_1,K_2)}(x)) &= d_{(K_1,K_2)}(e_{K_2}(e_{K_1}(x))) \\ &= d_{K_1}(d_{K_2}(e_{K_2}(e_{K_1}(x)))) \\ &= d_{K_1}(e_{K_1}(x)) \\ &= x. \end{aligned}$$